Rule versions v1 and v2

This note is the version record for the corpus request rule. It states exactly what changed between the two versions, why the change is the smallest one that resolves the counterexamples, and how the tooling keeps an old verdict from being read as current.

1. What a rule version is

A rule is a JSON document with a `rule_id` (stable across versions) and a `rule_version` (increment on every change to the document). The document is validated against `docs/schemas/rule.schema.json` (JSON Schema draft 2020-12) before it is used. Editing the rule and reloading it is what produces a new version; bumping the number without changing the document is a no-op, and reloading identical text does not mark the previous run outdated.

The rule declares:

- `scope`: which direction it speaks about (`A_to_B` here).
- `framing`: how the byte stream is cut into messages.
- `fields`: the offsets, types and, where known, the expected values.
- `hypothesis: true` on a field whose meaning is assumed, not observed.

2. The two documents side by side

```
// examples/corpus_rule_v1.json
{
  "schema_version": 1,
  "rule_id": "corpus_request",
  "rule_version": 1,
  "name": "corpus_request_v1",
  "scope": { "direction": "A_to_B" },
  "framing": {
    "type": "length_prefixed", "length_offset": 2, "length_size": 2,
    "byte_order": "big", "length_covers": "payload"
  },
  "fields": [
    { "name": "command", "offset": 0, "type": "uint8", "hypothesis": true, "expected": [1] },
    { "name": "flags", "offset": 1, "type": "uint8", "expected": [0] },
    { "name": "payload_length", "offset": 2, "type": "uint16", "byte_order": "big" },
    { "name": "target", "offset": 4, "type": "enum", "hypothesis": true, "enum": { "...":
16 } },
    { "name": "value", "offset": 5, "type": "uint16", "byte_order": "big",
      "hypothesis": true, "missing_ok": true }
  ]
}
```

v2 is the same document with one value changed:

```
   { "name": "command", "offset": 0, "type": "uint8", "hypothesis": true,
-    "expected": [1] }
+    "expected": [1, 2, 3] }
```

Nothing else differs. The framing is byte-identical, the field count is identical, every offset and type is identical. That is deliberate: the counterexamples pointed at one field, so one field changed.

3. Why this change and not another

Rule v1 produced eighty counterexamples, all at offset 0, all of the form "value 2 not in expected [1]" or "value 3 not in expected [1]". Three responses were available:

1. Widen `command` to `[1, 2, 3]`. The alternatives engine supports this: for `command` it reports `entropy_enum` with support 140 and contradict 0, and `constant` (the v1 reading) with support 60 and contradict 80. The evidence says the field is a small closed enum, not a constant.
2. Treat sessions `s2`/`s3` as a different rule and split the rule in two. This would have hidden the counterexamples without explaining them; the same layout and the same command position obtain in all three sessions, so a second rule would only restate the first with a different constant.
3. Leave the rule as v1 and record the boundary. This is what a cautious reading would do if the change had to be a guess. It was rejected because the evidence for an enum is direct, and because the rule marks `command` as a hypothesis, so widening it is the expected outcome of the check, not an overreach.

The chosen change is the narrowest of the three that removes the counterexamples without inventing structure.

4. Version differentiation

Running the CLI over the corpus reports, then diffing the two versions:

```
$ python -m src.protocol.cli verify --rule examples/corpus_rule_v1.json \
      --capture .../corpus_capture_01.normalized.json --out /tmp/v1.json
rule corpus_request v1: 280 messages, 80 counterexamples

$ python -m src.protocol.cli verify --rule examples/corpus_rule_v2.json \
      --capture .../corpus_capture_01.normalized.json --out /tmp/v2.json
rule corpus_request v2: 280 messages, 0 counterexamples

$ python -m src.protocol.cli diff --rule-a examples/corpus_rule_v1.json \
      --rule-b examples/corpus_rule_v2.json \
      --report-a /tmp/v1.json --report-b /tmp/v2.json
rule corpus_request: v1 -> v2
  ~ field command.expected: [1] -> [1, 2, 3]
report v1 -> v2
  resolved counterexamples: 80
  introduced counterexamples: 0
  unchanged mismatches: 0
  matched delta: +80
  coverage delta: +0.000
```

The rule diff names the exact path that changed and both values; the report diff says all eighty old counterexamples are resolved and none is introduced. The `matched delta` of +80 is the eighty mes-

sages that moved from mismatched to matched. `coverage delta` is unchanged because the `B_to_A` streams remain `not_applicable` in both versions: the scope did not move.

5. The old verdict is outdated, not current

When a rule is reloaded with a change, the engine keeps the previous application and marks every message in it `outdated`, with `reason: superseded_by_newer_rule`. `ResultStore` does the same for stored results: `mark_outdated(rule)` flags the results that belong to an older version of that rule. A report produced by v1 is therefore shown as v1's report, never as the current state, and the version comparison is the only place where v1 and v2 numbers are placed side by side.

6. What did not change

- The layout: command, flags, payload_length, target, value at the same offsets.
- The framing: length-prefixed at offset 2, two bytes, big-endian, length covers the payload, in both versions.
- The protocol's applicability: both versions scope to `A_to_B` and make no claim about `B_to_A`.
- The `synthetic_live` boundary: both versions match nothing there and report the same single `incomplete` message. Widening `command` to the corpus values did not widen the rule to the world.

7. Reproduce

```
python -m pytest tests/integration/test_reference_report.py \
    tests/hypothesis/test_versioning.py tests/protocol/test_rule.py -q
```